



JAVA PROGRAMMING

(340)

NATIONAL 2025

PRODUCTION:

Java National

_____ (540 points)

Test Time: 90 minutes

Solution and Project (There is NO partial credit) (NOTE: UC represents uppercase and LC represents lowercase)		
The Java source file is present on the flash drive in a single folder with your contest ID		10 points
Program Execution (If the program does not execute, then the remaining items in this section receive a score of zero)		
Menu appears when program starts		10 points
Menu UI Option 1 Print Chat_Bots: first 12 records are from ExperimentalLanguageBots.txt , last 10 are random from Supported_Languages.txt		30 points
Menu UI Option 1 Print Chat_Bots: Output format matches the sample provided		10 points
Menu UI Option 2 Highest Complex Chat_Bots: displays the Highest Complexity		20 points
Menu UI Option 2 Highest Complex Chat_Bots: Output format matches the sample provided		10 points
Menu UI Option 3 Least Complex Chat_Bots: displays the Least Complexity		20 points
Menu UI Option 3 Least Complex Chat_Bots: Output format matches the sample provided		10 points
Output 4 Complexity High to Low Range displays records in the proper order		20 points
Output 4 Complexity High to Low Range format matches the sample provided		10 points
Output 5 Complexity Low to High Range displays records in the proper order		20 points
Output 5 Complexity Low to High Range format matches the sample provided		10 points
Output 6 Primary Language Order (Low [A] to High Range [Z]) displays records in the proper order		20 points
Output 6 Primary Language Order (Low [A] to High Range [Z]) format matches the sample provided		10 points
Output 7 Primary Language Order (High [Z] to Low Range [A]) displays records in the proper order		20 points
Output 7 Primary Language Order (High [Z] to Low Range [A]) format matches the sample provided		10 points
Subtotal		/240 Points

Source Code Review No credit will be given for missing flagged comments. Code segments must work to receive full credit.		
A comment containing the contestant number is present at the top of the AIChatBotGenerator_National.java file		10 points
SC1: Flag Comment Present		5 points
SC1: AIChatBotGenerator_National.java class: comment data structures declaration and assignment statement of information gathered from ExperimentalLanguageBots.txt .		10 points
SC2: Flag Comment Present		5 points
SC2: AIChatBotGenerator_National.java class: comment data structures declaration and assignment statement of information gathered from Supported_Languages.txt .		10 points
SC3: Flag Comment Present		5 points
SC3: AIChatBotGenerator_National.java class: comment code file reading strategy was used for ExperimentalLanguageBots.txt using BufferedReader & readChatBotPremadeFrameworks()		15 points
SC4: Flag Comment Present		5 points
SC4: AIChatBotGenerator_National.java class readChatBotPremadeFrameworks() method: comment code demonstrating this method using BufferedReader readLine() method		15 points
SC5 A-C: Flag Comment Present		5 points
SC5 A-C: AIChatBotGenerator_National.java class readChatBotPremadeFrameworks() method: comment code //SC5A demonstrating finding delimiter (,) and //SC5B numerical data for integers and //SC5C for doubles from ExperimentalLanguageBots.txt		25 points
SC6: Flag Comment Present		5 points
SC6: AIChatBotGenerator_National.java class: comment code demonstrating ExperimentalLanguageBots.txt file NOT being found using try catch with exception being used		15 points
SC7: Flag Comment Present		5 points
SC7: AIChatBotGenerator_National.java class: comment code demonstrating file reading strategy was used for Supported_Languages.txt		15 points
SC8: Flag Comment Present		5 points
SC8: AIChatBotGenerator_National.java class readChatBotLanguages() method: comment code demonstrating gathering data from Supported_Languages.txt		15 points
SC9: Flag Comment Present		5 points
SC9: AIChatBotGenerator_National.java class readChatBotLanguages() method: comment code demonstrating removal of empty spaces from Supported_Languages.txt		15 points
SC10: Flag Comment Present		5 points
SC10: AIChatBotGenerator_National.java class: comment code with explanation of how to handle Supported_Languages.txt file NOT being found using try catch with an exception		15 points
SC11 A-B: Flag Comment Present		5 points
SC11 A-B: ChatBotReporter.java class method consoleMostComplex() comment code showing method was used by //SC11A menu and //SC11B implemented		10 points
SC12 A-B: Flag Comment Present		5 points
SC12 A-B: ChatBotReporter.java class method consoleLeastComplex() comment code showing method was used //SC12A menu and //SC12B implemented		10 points

SC13 A-B: Flag Comment Present		5 points
SC13 A-B: ChatBotReporter.java class method <i>consoleComplexRange_HIGH_to_low ()</i> comment code showing method was used //SC13A menu and //SC13B implemented		10 points
SC14 A-B: Flag Comment Present		5 points
SC14 A-B: ChatBotReporter.java class method <i>consoleComplexRange_LOW_to_high ()</i> comment code showing method was used //SC14A menu and //SC14B implemented		10 points
SC15 A-B: Flag Comment Present		5 points
SC15 A-B: ChatBotReporter.java class method <i>consoleLanguageRange_LOW_to_high ()</i> comment code showing method was used //SC15A menu and //SC15B implemented		10 points
SC16 A-B: Flag Comment Present		5 points
SC16 A-B: ChatBotReporter.java class method <i>consoleLanguageRange_HIGH_to_low ()</i> comment code showing method was used //SC16A menu and //SC16B implemented		10 points
Subtotal		/300 Points
Total Points		/540 Points

```

1
2 import java.io.FileNotFoundException;
3 import java.io.File;
4 import java.io.*;
5 import java.util.*;
6 import java.text.DecimalFormat;
7 import java.util.List;
8 import java.util.Scanner;
9
10
11 public class AIChatBotGenerator_National
12 {
13     //National Point
14     static Scanner sc = new Scanner(System.in);
15     static ArrayList<Chat_Bots> chat_Bots = new ArrayList<Chat_Bots>(); //SC2
16     static ArrayList<String> languagesComplete = new ArrayList<String>(); //SC1
17
18     public static void main (String args [])
19     {
20         //National Point to use BufferedReader to read this file
21         BufferedReader in = getReader("ExperimentalLanguageBots.txt");//SC3
22
23         Chat_Bots cB = readChatBotPremadeFrameworks(in); //SC3
24
25         chat_Bots.add(cB);
26         while (cB != null)
27         {
28             cB = readChatBotPremadeFrameworks(in); //SC3
29             if(cB != null)
30                 chat_Bots.add(cB);
31         }
32         //National Point to use Scanner to read this file. Use Try/Catch is points
33         try{ //SC10
34             readChatBotLanguages();
35
36         } catch (IOException e){
37             System.out.print("Error reading file Supported_Languages.txt");
38         }
39
40         setChat_Bots(10); //Given
41         ChatBotReporter.chatBotReporter(chat_Bots);
42     }
43
44
45
46
47
48     //Given
49     private static void setChat_Bots(int cB)
50     {
51         Random rand = new Random();
52         Chat_Bots s;
53
54         DecimalFormat df = new DecimalFormat("#.0000");
55         int languageCount = cB;
56         double complexity;
57         int tokenRate;
58         String primaryLanguage; String secondaryLanguage;
59
60         for(int i =0; i<languageCount; i++)
61         {
62
63             do{
64
65                 primaryLanguage = languagesComplete.get(rand.nextInt(languagesComplete.size()));
66                 secondaryLanguage = languagesComplete.get(rand.nextInt(languagesComplete.size()));
67             }while(primaryLanguage.equals(secondaryLanguage));
68             tokenRate = rand.nextInt(10)+11;

```

```

69         double tempComplexity = rand.nextDouble()*10.0 + 1.0;
70         String tempString =df.format(tempComplexity);
71         complexity = Double.parseDouble(tempString);
72         s = new Chat_Bots(primaryLanguage,secondaryLanguage,tokenRate,complexity);
73         chat_Bots.add(s);
74     }
75
76 }
77
78
79
80     //National Points
81     //National Points
82     private static Chat_Bots readChatBotPremadeFrameworks(BufferedReader in)
83     {
84         Chat_Bots cB;
85         String primaryLanguage; String secondaryLanguage; int tokenRate; Double complexity;
86         String line ="";
87         String[] data;
88
89         //catches errors in the read but not required since test files will not be corrupted
90         try
91         {
92             line = in.readLine(); //SC4
93
94         }
95         catch (IOException e)
96         {
97             System.out.println("I/O Error");
98             System.exit(0);
99         }
100
101         //ends the read once it gets to the end
102         if (line == null){
103
104             return null;
105         }
106         else
107         {
108             data = line.split(","); //SC5A //"," is the delimiter to separate the fields for the records
109             primaryLanguage = data[0].trim();
110             secondaryLanguage = data[1].trim();
111             tokenRate = Integer.parseInt(data[2].trim()); //SC5B
112             complexity = Double.parseDouble(data[3]); //SC5C
113
114         }
115         cB = new Chat_Bots(primaryLanguage,secondaryLanguage,tokenRate,complexity);
116
117         return cB;
118     }
119
120     //National Points
121     //use this method for Nationals. Must use Scanner and throws IOException (part of the points)
122     public static void readChatBotLanguages()throws IOException
123     {
124         Scanner sc2 = new Scanner(new File("Supported_Languages.txt")); //SC7
125         sc2.useDelimiter(","); //SC8
126
127         while(sc2.hasNext())//SC8
128         {
129             String tempLanguage = sc2.next().trim(); //SC9
130             languagesComplete.add(tempLanguage);
131         }
132     }
133 }
134 //National Points
135 private static BufferedReader getReader(String language)
136 {

```

```

137     BufferedReader in = null;
138     try //SC6
139     {
140         File file = new File(language);
141         in = new BufferedReader(
142             new FileReader(file) );
143     }
144     catch (FileNotFoundException e) //SC6
145     {
146         System.out.println(
147             "The ExperimentalLanguageBots.txt file doesn't exist.");
148         System.exit(0);
149     }
150     return in;
151 }
152
153 }
154
155 //////////////////////////////////////
156 //////////////////////////////////////
157 //////////////////////////////////////
158 //National Points. Get Points for Completing all of the required methods.
159 //However, they need to figure out how to make this work with the sort
160 class Chat_Bots
161 {
162     String primaryLanguage;
163     String secondaryLanguage;
164     int tokenRate_Level;
165     double complexity;
166
167     public Chat_Bots()
168     {
169         primaryLanguage = "Esperanto";
170         secondaryLanguage = "Java";
171         tokenRate_Level = 0;
172         complexity = 0.0;
173     }
174
175     public Chat_Bots(String primaryLanguage, String secondaryLanguage, int tR, double complexity)
176     {
177         this.primaryLanguage = primaryLanguage;
178         this.secondaryLanguage = secondaryLanguage;
179         this.tokenRate_Level = tR;
180         this.complexity = complexity;
181     }
182
183     public String getDefaultLanguages()
184     {
185         return secondaryLanguage + ", "+primaryLanguage;
186     }
187
188     public String getPrimaryLanguage()
189     {
190         return primaryLanguage;
191     }
192
193     public String getSecondaryLanguage()
194     {
195         return secondaryLanguage;
196     }
197
198     //Using an ArrayList Sort requires a Double wrapper
199     public Double getComplexity()
200     {
201         return (Double)complexity;
202     }
203
204     public int getTokenRate()
205     {
206         return tokenRate_Level;
207     }
208
209
210 }

```

NATIONAL KEY 2025

[illegible]

```

69     for (int i = 0; i < chat_Bots.size(); i++) {
70         Chat_Bots currentBot = chat_Bots.get(i);
71         if (!currentBot.getPrimaryLanguage().startsWith("_") && currentBot.getComplexity() > highestComplexity)
{
72             index = i;
73             highestComplexity = currentBot.getComplexity();
74         }
75     }
76
77     if (index != -1) {
78         Chat_Bots highestComplexBot = chat_Bots.get(index);
79         System.out.println("Highest Complexity: " + highestComplexity + " " + highestComplexBot.getDefaultLangua
ges());
80     } else {
81         System.out.println("No chat bot with valid complexity found.");
82     }
83
84     System.out.println("-----");
85     System.out.println("\n");
86 }
87
88 private static void consoleLeastComplex(List<Chat_Bots> chat_Bots) { //SC12B
89     // Your implementation for consoleLeastComplex method here
90     System.out.println("\n");
91     System.out.println("-----Lowest Complexity-----");
92     int index = -1;
93     double leastComplexity = Double.MAX_VALUE;
94
95     for (int i = 0; i < chat_Bots.size(); i++) {
96         if (!chat_Bots.get(i).getPrimaryLanguage().startsWith("_") && chat_Bots.get(i).getComplexity() < leastCo
mplexity) {
97             index = i;
98             leastComplexity = chat_Bots.get(i).getComplexity();
99         }
100     }
101
102     if (index != -1) {
103         System.out.println("Least Complexity: " + chat_Bots.get(index).getComplexity() + " " + chat_Bots.get(ind
ex).getDefaultLanguages());
104     } else {
105         System.out.println("No chat_Bots found with valid complexity.");
106     }
107     System.out.println("-----");
108     System.out.println("\n");
109 }
110
111 private static void consoleComplexRange_HIGH_to_low(List<Chat_Bots> chat_Bots) { //SC13B
112     // Your implementation for consoleComplexRange_HIGH_to_low method here
113     System.out.println("\n");
114
115     System.out.println("-----Complexity High to Low Range-----");
116     chat_Bots.sort((o1, o2)-> o2.getComplexity().compareTo(
117         o1.getComplexity()));
118     printChat_Bots(chat_Bots);
119     System.out.println("-----");
120     System.out.println("\n");
121 }
122
123 private static void consoleComplexRange_LOW_to_high(List<Chat_Bots> chat_Bots) { //SC14B
124     // Your implementation for consoleComplexRange_LOW_to_high method here
125     System.out.println("\n");
126     System.out.println("-----Complexity Low to High Range-----");
127     chat_Bots.sort((o1, o2)-> o1.getComplexity().compareTo(
128         o2.getComplexity()));
129     printChat_Bots(chat_Bots);
130     System.out.println("-----");
131     System.out.println("\n");
132 }

```

```

133
134     private static void consoleLanguageRange_LOW_to_high(List<Chat_Bots> chat_Bots) { //SC15B
135         // Your implementation for consoleLanguageRange_LOW_to_high method here
136         System.out.println("\n");
137         System.out.println("-----Primary Language Order (Low [A] to High Range [Z])-----");
138         chat_Bots.sort((o1, o2)-> o1.getSecondaryLanguage().compareTo(
139             o2.getSecondaryLanguage()));
140         printChat_Bots(chat_Bots);
141         System.out.println("-----");
142         System.out.println("\n");
143     }
144
145     private static void consoleLanguageRange_HIGH_to_low(List<Chat_Bots> chat_Bots) { //SC16B
146         // Your implementation for consoleLanguageRange_HIGH_to_low method here
147         System.out.println("\n");
148         System.out.println("-----Primary Language Order (High [Z] to Low Range [A])-----");
149         chat_Bots.sort((o1, o2)-> o2.getSecondaryLanguage().compareTo(
150             o1.getSecondaryLanguage()));
151         printChat_Bots(chat_Bots);
152     }
153     /*private static void printChat_Bots(List<Chat_Bots> chat_Bots){
154         int i = 1;
155         for (Chat_Bots cB : chat_Bots)
156         {
157             System.out.println(i+" "+cB.getDefaultLanguages() + " | Token Rate: "+cB.getTokenRate()+" | Complexity:
158             "+cB.getComplexity());
159             i++;
160         }
161     }
162     */
163     private static void printChat_Bots(List<Chat_Bots> chat_Bots) {
164         int i = 1;
165         for (Chat_Bots cB : chat_Bots) {
166             System.out.print("Chat Bot " + i + ": ");
167             System.out.print("Default Languages: " + cB.getDefaultLanguages() + " | ");
168             System.out.print("Token Rate: " + cB.getTokenRate() + " | ");
169             System.out.println("Complexity: " + cB.getComplexity());
170             i++;
171         }
172     }
173 }
174 }
175

```